

Agent 记忆全景综述：20+顶尖机构联合出品，Agent memory看这一篇就够了

原创 Victor 硅基捕手维克托 2026年3月26日 09:48 新加坡

Rethinking Memory Mechanisms of Foundation Agents in the Second Half: A Survey

Wei-Chieh Huang^{1,†}, Weizhi Zhang^{1,†,‡}, Yueqing Liang^{2,†}, Yuanchen Bei^{3,*}, Yankai Chen^{1,18,*}, Tao Feng^{3,*}, Xinyu Pan^{4,*}, Zhen Tan^{5,*}, Yu Wang^{14,*}, Tianxin Wei^{3,*}, Shanglin Wu^{6,*}, Ruiyao Xu^{7,*}, Liangwei Yang^{22,*}, Rui Yang^{3,*}, Woosong Yang^{1,*}, Chin-Yuan Yeh^{1,*}, Hanrong Zhang^{1,*}, Haozhen Zhang^{8,*}, Siqi Zhu^{3,*}, Henry Peng Zou¹, Wanjia Zhao²¹, Song Wang⁹, Wujiang Xu¹⁰, Zixuan Ke²², Zheng Hui¹¹, Dawei Li⁵, Yaozu Wu¹³, Langzhou He¹, Chen Wang¹, Xiong Xiao Xu², Baixiang Huang⁶, Juntao Tan²², Shelby Heinecke²², Huan Wang²², Caiming Xiong²², Ahmed Abdelhadi Metwally²³, Jun Yan²³, Chen-Yu Lee²³, Hanqing Zeng²⁴, Yinglong Xia²⁴, Xiaokai Wei²⁵, Ali Payani²⁶, Yu Wang²⁷, Haitong Ma¹², Wenya Wang⁸, Chenguang Wang¹⁵, Yu Zhang¹⁶, Xin Eric Wang¹⁷, Yongfeng Zhang¹⁰, Jiaxuan You³, Hanghang Tong³, Xiao Luo⁴, Xue Steve Liu^{18,19}, Yizhou Sun²⁰, Wei Wang²⁰, Julian McAuley¹⁴, James Zou²¹, Jiawei Han³, Philip S. Yu^{1,¶}, Kai Shu^{6,¶}

[†] Co-first authors * Core contributors (listed alphabetically) [‡] Project organizer [¶] Core supervisors


AgentMemoryWorld/Awesome-Agent-Memory 公众号 · 硅基捕手维克托

论文链接: <https://arxiv.org/abs/2602.06052>

发表时间: 2026.02.10

机构: 伊利诺伊、Meta、Google 等 20+ 机构

用 GPT 或 Claude 做过长对话的人大概都踩过这个坑: 聊了半个小时, AI 把你前面说过的事情忘干净了。你不得不把背景重新解释一遍。

这还是人机对话, 忍一忍也就算了。

但如果是 agent 在自主执行任务呢? 记不住"这个 API 上次试过了、失败了", 每次重新出发就是在浪费资源, 会无限踩同一个坑。

这个问题在 AI 进入"下半场"之后变得格外突出。

这篇综述动用了将近 50 位作者, 涵盖 20 多家研究机构, 覆盖近年来发布的数百篇论文, 试图把 agent 记忆这件事完整说清楚。

看这个作者和机构数量, 就知道这篇工作的工作量了。

综述也统计了, 从 2025 年下半年开始, 针对记忆的研究在快速增长。

上半场和下半场

论文的出发点是一个判断：AI 研究正在经历范式转移。

上半场在做什么？刷 benchmark。模型越来越大、分数越来越高，大家都在比谁能在静态数据集上跑出更好的数字。

下半场要面对的是另一类问题。真实的 agent 要在长时程、动态、依赖用户的环境里工作。任务可能跨越几十次交互，环境随时在变，用户的偏好也各不相同。

这种场景下，"上下文爆炸"是个真实的工程问题。agent 需要不断积累、管理、有选择地复用大量信息，这些信息远远塞不进一个 context window。

记忆，就是填补这个效用鸿沟的核心机制。

三个维度，一套统一的框架

这篇综述的主要贡献是提出了一个三维分类框架，把之前零散的记忆研究统一起来。



第一维：记忆存在哪里（Memory Substrate）

外部记忆是显式存储在模型权重之外的信息。常见形式有：

- 向量数据库：RAG 的主力，用近似最近邻搜索检索嵌入向量
- 结构化存储：图结构、关系表、层级树，典型系统有 AriGraph、HippoRAG、RAPTOR
- 文本记录：人类可读的摘要和时序日志，2023 年的 Generative Agents 就是这条路线

内部记忆是编码在模型本身里的：

- 权重：训练出来的参数化知识，相当于"写死"在模型里的记忆
- 上下文窗口：运行时的工作区，存放 prompt、推理链、工具输出
- KV Cache：推理过程中的中间状态

这两类存储各有取舍。外部记忆容量大、可编辑、可检查，但检索有延迟。内部记忆访问快，但修改成本高、容量有限。

第二维：认知机制（Cognitive Mechanism）

这是论文最有意思的部分。作者借鉴认知科学里对人类记忆的分类，把 agent 记忆拆成五种：

感知记忆（Sensory Memory）

最短暂的那一层。作用是缓冲输入信号，在进一步处理前做短暂保留。在文本 agent 里几乎不需要。

但在多模态和具身 agent 里越来越重要。典型用途是保留最近几秒的视频帧或传感器嵌入，用来平滑感知、应对短暂遮挡。代表系统有 HMT、LightMem、SAM2。

工作记忆 (Working Memory)

"当前正在处理什么"。对应的就是 agent 的 context window，是有容量约束的在线工作区。

以前大家把它当被动缓冲区，塞满了就截断或压缩。这篇综述把它重新定义为一个需要主动管理的计算资源。

两条研究线：一是写入前的整形，在信息进入上下文前先做压缩、折叠、抽象，Context-Folding 是代表。二是在线淘汰和更新，MemGPT 的分页机制走的是这条路。

情节记忆 (Episodic Memory)

"发生过什么"。跨会话的具体经历记录，带时间和情境上下文。比如"上次你偏好两页摘要"、"上个方案因为缺 API key 失败了"。

难点是两个：怎么存，包括什么时候触发存储、存多少颗粒度；怎么取，也就是在需要时检索出相关的历史片段。代表系统有 Memoria、GCAgent。

语义记忆 (Semantic Memory)

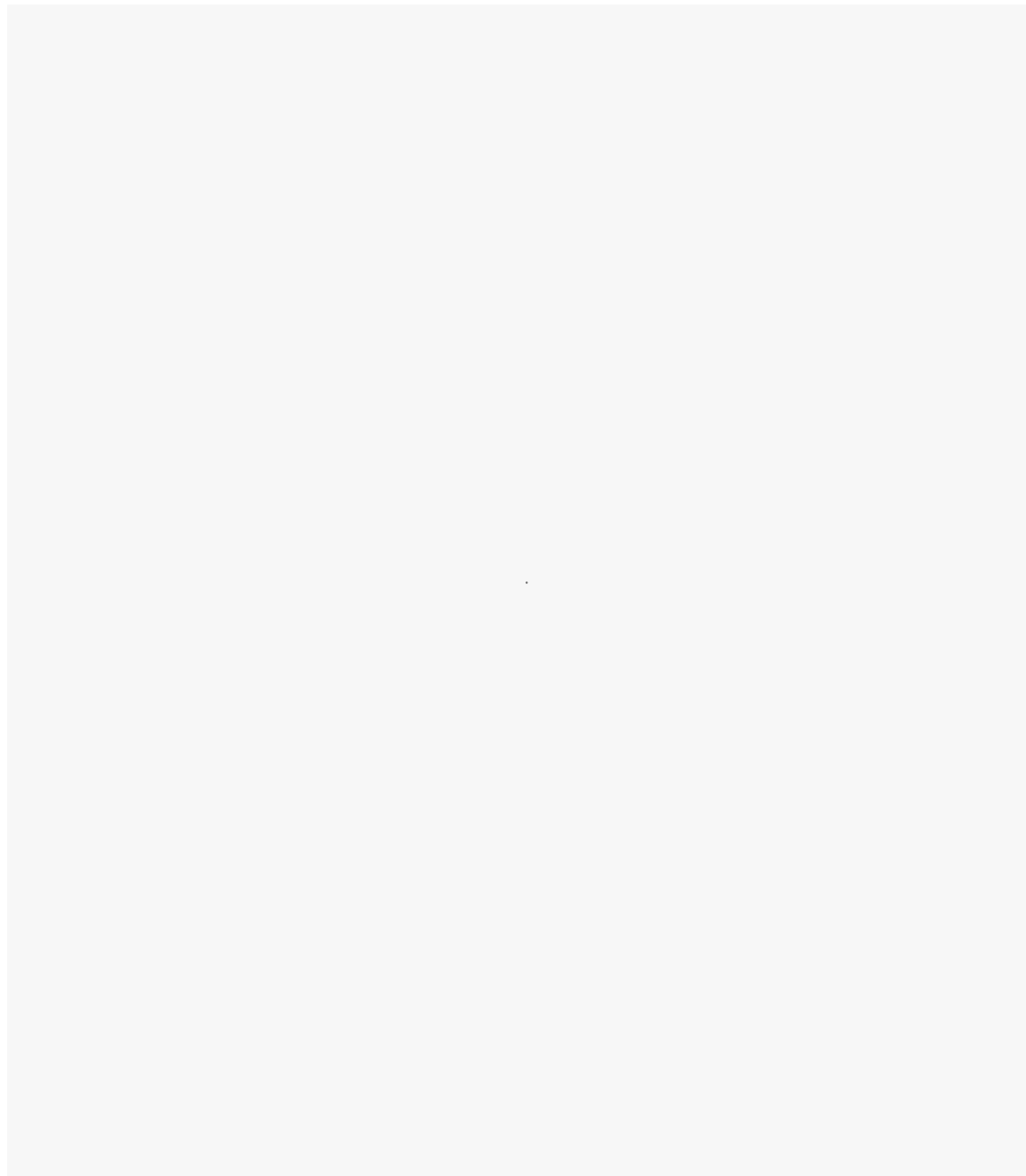
"知道什么"。从情节中蒸馏出来的稳定概念和事实知识。不是某次具体经历，而是从多次经历中归纳出的通用知识。

实现形式包括知识图谱、层级 schema、辅助参数等。代表系统有 HippoRAG、Zep、Titans。

程序记忆 (Procedural Memory)

"怎么做事"。可复用的技能、动作模式和工作流。比如"搜索 → 阅读 → 提取 → 引用"这个固定流程被固化成一个可调用的 routine。

形式上从非参数化模板到参数化神经策略都有，前者是把 workflow 写下来，后者是通过强化学习训练成"本能"。代表系统有 Voyager、Agent Workflow Memory、LEGOMem。



第三维：记忆服务谁（Memory Subject）

- **用户中心记忆**：记录特定用户的偏好、背景、交互历史，用于个性化
- **Agent 中心记忆**：记录 agent 自身的任务经验、技能、领域知识，用于跨任务的能力积累

这两类技术上有交叉，但目标和评估维度不同。工作记忆、感知记忆和程序记忆基本上是 agent 中心的。语义记忆和情节记忆在两类场景里都会出现。

论文用了一张散点图来呈现这个分布关系：五种记忆类型在 agent 中心和用户中心两个方向上，各自的研究论文数量差异明显，感知记忆几乎全是 agent 中心，语义记忆则在两侧都很活跃。

记忆怎么运转：五种核心操作

有了记忆系统，agent 怎么用它？论文把单 agent 的记忆操作归纳为五类，覆盖记忆的完整生命周期。

存储与索引（Storage and Index）

信息以什么形式写入、怎么组织，直接决定后续检索的精度和效率。向量嵌入加上时间戳、任务标识符等辅助元数据是主流做法。

结构化存储格式支持更复杂的关系查询，图、关系表、层级树都有各自的适用场景。存储格式的选择会一路影响到推理质量。

加载与检索（Loading and Retrieval）

从存储里把相关记忆取出来，注入当前推理过程。核心挑战是平衡相关性、多样性和上下文预算。

取太多是噪声，取太少是遗漏。Pre-filtering 先按元数据筛一遍，再做语义相似度排序，是比较常见的两阶段做法。

更新与刷新（Updates and Refresh）

记忆不是写进去就不变的。任务完成后、检测到不一致时，需要重写语义摘要、合并重叠的情节记录、调整重要性评分。Reflexion 里的反思机制就属于这类，通过自我评估触发记忆更新。

压缩与摘要（Compression and Summarization）

把细粒度的情节记录转化成紧凑的抽象表示。RAPTOR 的分层压缩方案把记忆组织成多级树状结构，不同颗粒度满足不同检索需求。Dynamic Cheatsheet 方案则维护一个持续更新的任务摘要，避免每次都做大规模检索。

核心取舍是抽象保真度和长期可回溯性之间的矛盾——压得越狠，细节丢得越多。

遗忘与保留（Forgetting and Retention）

不加筛选地积累记忆，迟早会影响推理质量。简单策略是按时间衰减或重要性阈值淘汰，进阶做法是用 RL 学习记忆保留策略，BudgetMem 和 Memory-R1 走的是这条路。遗忘不是失败，是维持记忆系统健康运转的必要机制。

多 Agent 的记忆拓扑

多个 agent 协作时，记忆怎么共享和隔离？论文归纳了四种架构：

私有隔离 (Private-Only)：每个 agent 的记忆完全独立，互不干扰。隐私保护好，但会产生大量冗余记忆。

共享工作区 (Shared-Workspace)：所有 agent 共用一个记忆池，读写权限都有。通信成本低，但容易产生噪声和冲突。MetaGPT 的共享制品池就是这条路线。

混合架构 (Hybrid)：既有私有层，也有共享层，由策略决定什么信息放哪里。Collaborative Memory 和 MirrorMind 是代表。

编排架构 (Orchestrated)：有一个显式的控制器负责任务分解和记忆访问的中介。ChatDev、MIRIX 属于这类，流程清晰，但控制器本身容易成为瓶颈。

记忆怎么学：三种策略

Agent 怎么建立和改进记忆操作的策略？论文把这部分单独成章。

基于 Prompt (Prompt-Based)：静态或动态 prompt，告诉模型什么时候存、存什么、怎么检索。不需要训练，灵活，但效果上限低。

监督微调 (SFT)：通过标注数据训练记忆的参数化、检索质量和稳定性，泛化性更好，但需要数据。

强化学习 (RL)：按照奖励信号优化记忆操作策略，可以做步骤级、轨迹级、跨 episode 的学习。论文认为这是最有前景的方向。

MEM1 是 RL 这条路线的代表，一个端到端的框架，让 agent 在有界记忆预算下完成任意长度的交互任务。内存使用接近常数，不随交互轮数线性增长。

Mem-α 也走类似路线。这两个系统指向同一个结论：记忆大小不应该和交互轮数成正比。

记忆在实际场景里怎么用

论文梳理了记忆机制在不同领域的落地方式。我挑几个比较有代表性的：

科研辅助

科研 agent 需要综合大量文献、维护多阶段推理的来龙去脉。IterResearch 用 Markovian 状态重建方案，只保留不断演化的报告和最新结果，避免上下文被历史信息撑爆。MirrorMind 模拟集体智能，从层级化的认知风格库和知识库检索特定视角。

软件工程

有效的代码 agent 不只是生成代码，还要回忆之前的失败轨迹。MetaGPT 把程序记忆用于开发 workflow，把语义记忆用于项目规范。SWE-Bench 的实验显示，记忆机制显著提升了多文件跨 issue 的修复成功率。

对话系统

MemGPT 把 context window 当操作系统来管，在主 context 和外部存储之间做数据调度。更新的 O-Mem 则维护一个三组件的用户画像，从交互中持续提取和更新用户 persona。

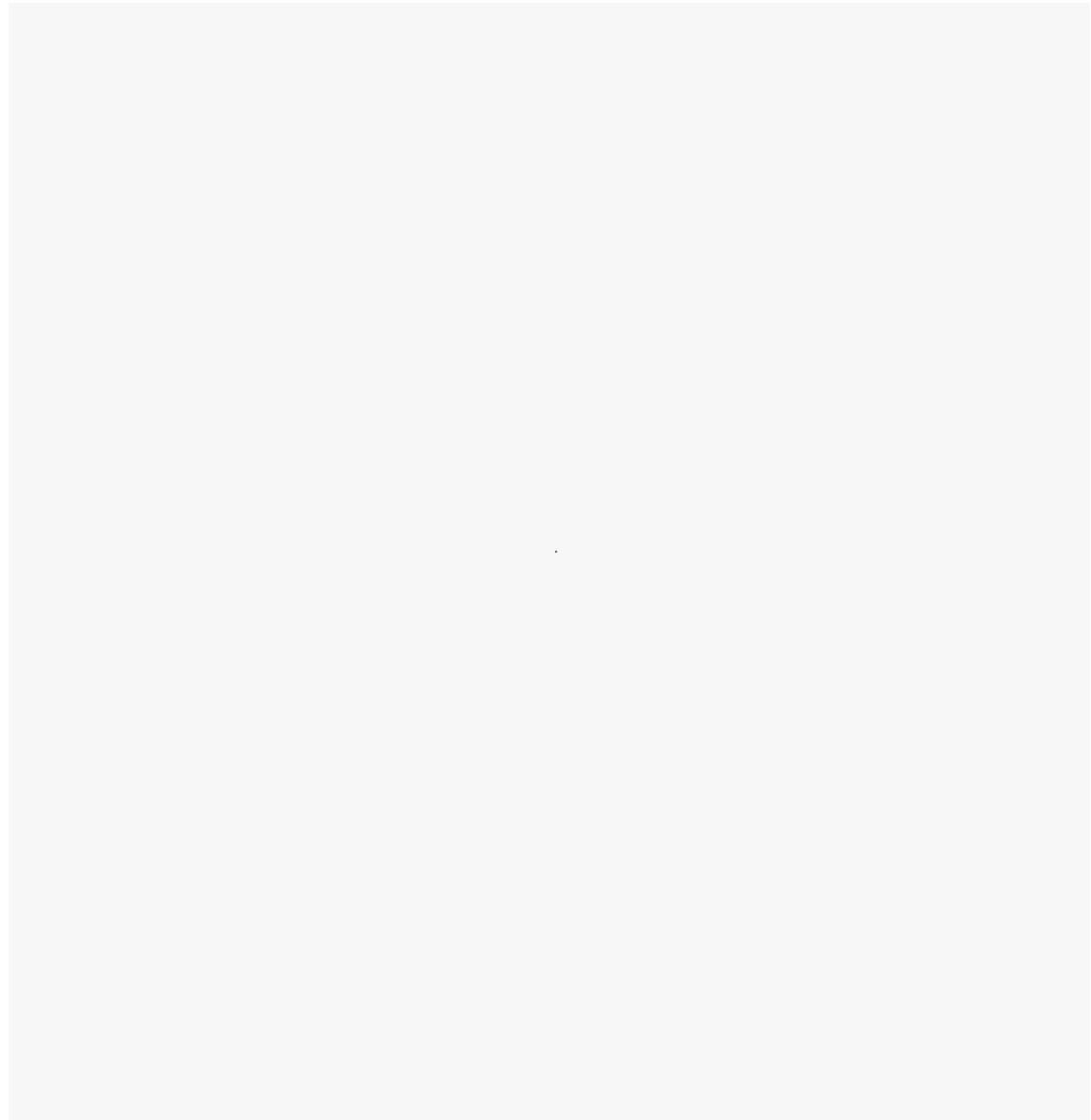
机器人与具身智能

Memo 用周期性摘要 token 压缩轨迹信息用于长视野导航，通过 RL 训练。MG-Nav 用地标区域而非稠密点云构建空间记忆图，模拟人类导航的工作方式。记忆在这里充当高层规划与低层控制之间的

桥梁。

教育

LOOM 构建学习者记忆图，映射教学概念之间的前置依赖关系。Agent4Edu 直接复现艾宾浩斯遗忘曲线来模拟知识衰减，用于教师培训。这里记忆不只是历史日志，更接近学生的"认知数字孪生"。



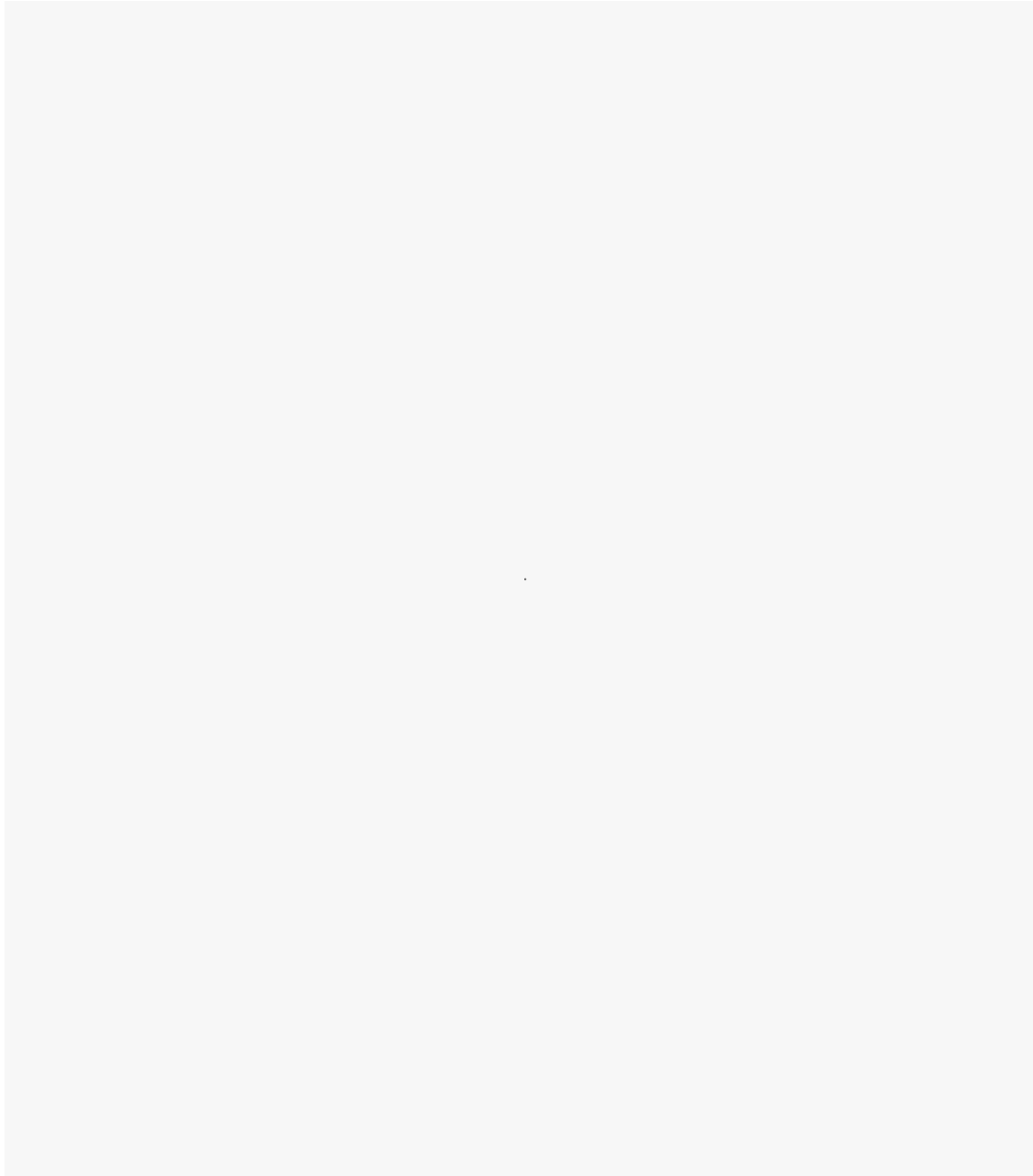
怎么评估记忆效果

论文整理了两类 benchmark：用户中心和 agent 中心。

用户中心的 benchmark 里，LongMemEval 比较全面，50K 会话、500 个问题、最长 150 万 token。HaluMem 专注于记忆幻觉，设计了记忆完整性（Memory Integrity, MI）和虚假记忆率（False Memory Rate, FMR）两个专项指标。这是我看到比较有针对性的评估设计。



Agent 中心方面，SWE-Bench 和 WebArena 是比较常见的基准，前者 2.3K 实例考察代码 patch 解决率，后者 812 个 web 任务考察成功率。OSWorld 包含 369 个多模态 OS 任务，也开始被引用。



论文直接指出了现有 benchmark 的系统性缺陷：几乎所有评估都是重置式的、孤立的、短时程的。它们不考察 agent 是否真的在跨任务积累知识，也不考察长期一致性。这是个真实的评估空白。

六个悬而未决的问题

论文最后列了六个方向，我觉得最有意思的三个：

持续学习与自进化：现在的 agent 在单次任务里还行，但跨任务的知识迁移基本是空白。需要把情节、语义、程序记忆整合进 post-training 流程，而不是靠推理时的启发式策略。

记忆基础设施与效率：论文把进化路径描述为三级，分别是有组织的文本记忆、压缩的隐向量记忆、通过 RL 吸收进模型状态的参数化内化记忆。当前大多数系统停在第一级。

可信记忆与隐私：记忆越强，攻击面越大。记忆投毒、对抗性记忆注入、黑盒提取攻击已经是真实的威胁向量，相关攻击工作在 2024-2025 年间陆续有论文发表。用户对记忆内容的可视、可编辑、可撤销控制，目前几乎没有系统认真做。



我的感受

读这篇综述之前，我对 agent 记忆的理解停留在"加个 RAG 就行"这个层次。读完发现这件事的纵深远超我预期。

光是"工作记忆应该被主动管理而不是被动截断"这一个观点，就足以改变我写 agent prompt 的方式。

程序记忆从 workflow 模板向 RL 训练策略的演进，也让我开始重新思考"agent 技能"到底应该用什么形式存储。

目前这个领域的评估是真正的瓶颈。大家都在孤立的静态 benchmark 上跑分。但真正有价值的 agent 是那种能跨任务积累经验、越用越聪明的。这个特性在现有 benchmark 里基本无法被衡量。在有更好的评估框架出现之前，很多"记忆能力提升"的声称其实很难被认真验证。



硅基捕手维克托

在硅基荒原，捕捉AI进化的微光 | 最新大模型及Agent技术分享

51篇原创内容

公众号

AI Agent · 目录 ≡

<| 上一篇

SkillNet：给技能建了个靠谱的技能市场

下一篇 >

长文总结：近一年 Agent 自进化的两大方向和四大趋势

